



Virtual Memory

Week 5: Penjelasan Step-by-Step

informatika-itera.github.io/os

Program Studi Teknik Informatika
Institut Teknologi Sumatera

2026

OUTLINE

Peta materi pertemuan ini

1. **Mengapa Virtual Memory?**
2. **API Memori (C) Secara Praktis**
3. **Mekanisme Translasi Alamat**
4. **Segmentasi**
5. **Manajemen Free-Space**
6. **Ringkasan dan Latihan**

Capaian Pembelajaran

Tujuan setelah pertemuan selesai

1. Menjelaskan kenapa virtual memory dibutuhkan.
2. Memahami address space dengan contoh sederhana.
3. Menelusuri translasi alamat langkah demi langkah.
4. Memahami segmentasi dan masalah fragmentasi.
5. Memahami dasar allocator: split, coalesce, fit policy.

Fokus Kelas Ini

Kita belajar pelan-pelan, **dari konsep paling dasar** sampai mekanisme yang dipakai OS modern.

Cara Belajar Hari Ini

Supaya materi tidak terasa berat

1. Lihat dulu masalahnya (kenapa VM perlu).
2. Pahami istilah satu per satu.
3. Ikuti contoh hitung sederhana.
4. Cek pemahaman lewat pertanyaan pendek.
5. Ulangi dengan analogi kehidupan sehari-hari.

Aturan Main Kelas

Jika ada satu langkah belum paham, jangan lanjut dulu ke langkah berikutnya.

Masalah Dasar Tanpa Virtual Memory

Apa yang rusak jika semua proses menyentuh RAM langsung?

- Proses A bisa menimpa data proses B.
- Programmer harus tahu alamat fisik asli (sulit sekali).
- Sulit memindah proses saat sistem ramai.

Inti Masalah

Banyak proses, satu RAM fisik. Bagaimana caranya tetap aman, adil, dan mudah diprogram?

Analogi Sederhana: Apartemen

Cara mudah membayangkan virtual memory

Bayangkan RAM seperti apartemen besar:

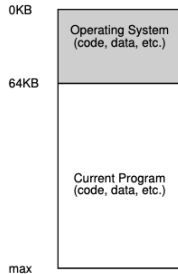
1. Banyak penghuni (proses) tinggal di satu gedung (RAM fisik).
2. Tiap penghuni merasa punya unit sendiri (address space virtual).
3. Satpam dan pengelola gedung (hardware + OS) mencegah pelanggaran.

Makna Analogi

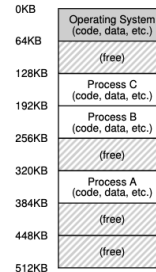
Virtual memory = ilusi yang terkontrol. Ilusi ini membuat hidup programmer jauh lebih mudah.

Dari Sistem Lama ke Multiprogramming

Perubahan kebutuhan dari 1 proses ke banyak proses



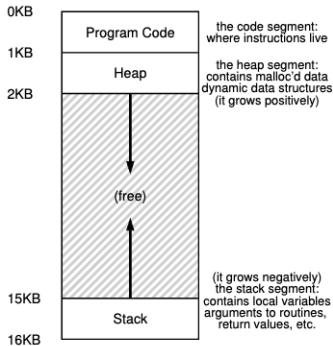
Sistem awal: OS + satu program, sederhana tapi tidak skalabel.



Multiprogramming: banyak proses aktif, butuh isolasi dan proteksi.

Address Space Proses

Struktur memori yang dilihat program



- **Code**: instruksi program.
- **Data**: variabel global/statis.
- **Heap**: alokasi dinamis, cenderung tumbuh ke atas.
- **Stack**: frame fungsi, cenderung tumbuh ke bawah.

Program hanya melihat peta virtual ini. OS yang menghubungkan ke RAM fisik.

Checkpoint 1

Cek pemahaman sebelum lanjut

1. Kenapa proses A tidak boleh menulis ke memori proses B?
2. Kenapa programmer sebaiknya tidak mengatur alamat fisik langsung?
3. Apa bedanya RAM fisik dengan address space virtual?

Kunci Jawaban Singkat

VM ada untuk **isolasi**, **kemudahan**, dan **fleksibilitas**.

Stack vs Heap (Pelan-Pelan)

Ini pondasi sebelum bicara allocator

Aspek	Stack	Heap
Cara alokasi	Otomatis saat fungsi dipanggil	Manual lewat <code>malloc()</code>
Kapan hilang	Otomatis saat fungsi selesai	Harus dipanggil <code>free()</code>
Cepat/lambat	Umumnya lebih cepat	Lebih fleksibel, tapi perlu pengelolaan
Risiko bug	Stack overflow	Leak, double free, dangling pointer

Langkah Aman Pakai `malloc/free`

Urutan kerja yang sebaiknya selalu diikuti

1. Tentukan ukuran data yang benar.
2. Panggil `malloc(size)`.
3. Cek apakah hasilnya `NULL`.
4. Gunakan memori sesuai batas ukuran.
5. Panggil `free(ptr)` saat selesai.
6. Set pointer menjadi `NULL`.

Kenapa langkah 6 penting?

Supaya pointer lama tidak dipakai lagi secara tidak sengaja (mencegah dangling pointer).

Bug 1: Memory Leak dan Double Free

Dua bug klasik yang sering muncul

Memory Leak

- Terjadi saat memori sudah diminta, tapi tidak pernah dikembalikan.
- Dampak: aplikasi makin lama makin boros RAM.

Double Free

- Terjadi saat pointer yang sama di-`free()` lebih dari sekali.
- Dampak: struktur allocator bisa rusak, program crash.

Bug 2: Dangling Pointer dan Out-of-Bounds

Bug yang sering sulit dideteksi

Dangling Pointer

- Pointer masih dipakai padahal memorinya sudah dibebaskan.

Out-of-Bounds

- Menulis lewat batas buffer (misal index terlalu besar).
- Bisa merusak data lain di memori.

Checklist Cepat

Validasi ukuran, validasi index, dan disiplin pada siklus hidup pointer.

Checkpoint 2

Pastikan pondasi API memori sudah kuat

1. Apa beda umur data pada stack dan heap?
2. Kenapa `free()` wajib dipanggil?
3. Kenapa pointer sebaiknya diset ke `NULL` setelah `free()`?

Video Pendukung (API Memori C)

Cocok ditonton di rumah

How to manage memory with malloc, calloc, realloc, and free in C

<https://www.youtube.com/watch?v=lQP4X3odvHE> Fokus pada pola aman `malloc()` → pakai sesuai batas → `free()`.

Apa Itu Translasi Alamat?

Inti kerja VM di level hardware

Saat program mengakses alamat, yang keluar dari CPU adalah **alamat virtual**, bukan alamat fisik.

Hardware Memory Management Unit (MMU) melakukan:

1. Cek valid/tidaknya alamat.
2. Hitung alamat fisik yang sebenarnya.
3. Raise exception jika akses tidak legal.

Base-and-Bounds: Konsep Dasar

Model translasi yang paling mudah dipahami

Dua register penting:

- **Base**: titik awal memori fisik proses.
- **Bounds**: ukuran area legal proses.

Rumus saat valid:

$$\text{Physical Address} = \text{Base} + \text{Virtual Address}$$

Jika tidak valid (melewati bounds), maka trap ke OS.

Translasi Langkah Demi Langkah

Step 1 sampai Step 4

1. Proses meminta alamat virtual V .
2. Hardware cek: $V < \text{bounds}$?
3. Jika benar: hitung $\text{base} + V$.
4. Jika salah: kirim exception ke kernel.

Makna: proteksi berjalan otomatis, tidak menunggu aplikasi sadar salah.



Contoh Hitung Translasi

Latihan angka sederhana

Diketahui `base = 4000`, `bounds = 1000`.

V	Valid?	Alamat Fisik
120	Ya	4120
980	Ya	4980
1200	Tidak	Trap

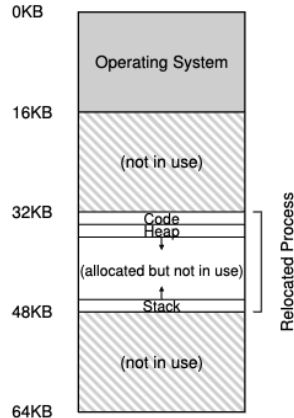
Ingat

v boleh kecil, tapi tetap harus dicek terhadap `bounds` setiap akses.

Apa yang Terjadi Saat Trap?

Alur saat proses melanggar batas

1. Hardware mendeteksi akses ilegal.
2. Hardware menghentikan sementara proses user.
3. Kontrol pindah ke kernel (exception handler).
4. OS memutuskan tindakan: hentikan proses, kirim sinyal error, dll.



Kerja Sama Hardware dan OS

Mekanisme cepat + kebijakan aman

Komponen	Peran
Hardware	Translasi alamat, cek bounds, dan trap secara cepat.
Hardware	Menyediakan mode privileged untuk instruksi sensitif.
OS	Mengisi register base/bounds saat context switch.
OS	Menyiapkan dan menjalankan handler exception.
OS	Mengalokasikan dan membebaskan memori proses.

Checkpoint 3

Evaluasi cepat bagian translasi

1. Kenapa translasi sebaiknya dilakukan hardware, bukan software murni?
2. Kapan trap terjadi pada base-and-bounds?
3. Saat context switch, kenapa register base/bounds harus diperbarui?

Video Pendukung (Translasi Alamat)

Cocok ditonton di rumah

1. Mechanism of Address Translation

<https://www.youtube.com/watch?v=mYxkp5vpyqc>

Mengapa Segmentasi Diperlukan?

Kelemahan model satu blok kontigu

Jika satu address space diperlakukan sebagai satu blok:

- Banyak ruang kosong ikut terbawa ke RAM fisik.
- Pemakaian RAM jadi boros.

Segmentasi membagi address space menjadi bagian logis:

- Code
- Heap
- Stack

Cara Membaca Alamat pada Segmentasi

Langkah 1 sampai 4

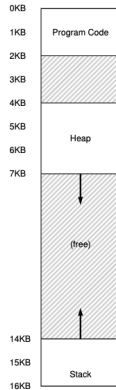
1. Ambil bit atas untuk menentukan nomor segmen.
2. Ambil sisa bit sebagai offset dalam segmen itu.
3. Cek offset apakah masih dalam size segmen.
4. Jika valid, hitung alamat fisik = base segmen + offset.

Keuntungan

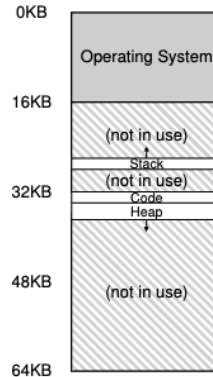
Tiap segmen bisa dipindah atau diatur sendiri, jadi lebih fleksibel daripada satu blok besar.

Contoh Segmentasi di Memori Fisik

Visual kiri-kanan: ruang virtual vs penempatan fisik



(Kiri) Address space proses dengan celah ruang.



(Kanan) Segmen ditempatkan terpisah di RAM fisik.

Contoh Nilai Register Segmen

Supaya tidak abstrak

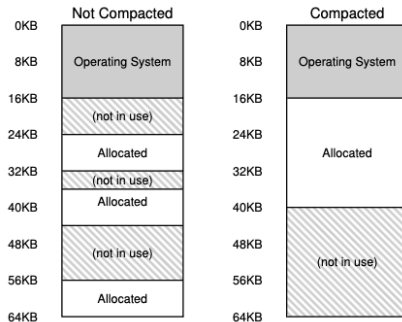
Segment	Base	Size
Code	32K	2K
Heap	34K	3K
Stack	28K	2K

Contoh cepat: jika offset heap = 1K, alamat fisik heap = $34K + 1K = 35K$ (jika masih dalam size).

Fragmentasi Eksternal dan Compaction

Masalah utama pada segmentasi

- Ruang kosong total bisa banyak.
- Tapi ruang itu tersebar jadi potongan kecil.
- Request besar bisa gagal meskipun total kosong cukup.
- Compaction menggabungkan ruang kosong, tapi mahal.



Checkpoint 4

Pastikan segmentasi sudah masuk

1. Kenapa segmentasi bisa lebih hemat dibanding satu blok besar?
2. Kenapa tetap bisa muncul fragmentasi eksternal?
3. Kenapa compaction tidak dilakukan terus-menerus?

Video Pendukung (Segmentasi)

Cocok ditonton di rumah

Operating Systems Lecture: Main Memory (Segmentation)

<https://www.youtube.com/watch?v=akADkYH5M8o> Cocok untuk menguatkan konsep *base/size* per segmen.

Masalah di Allocator

Permintaan ukuran memori berubah-ubah

Tantangan allocator:

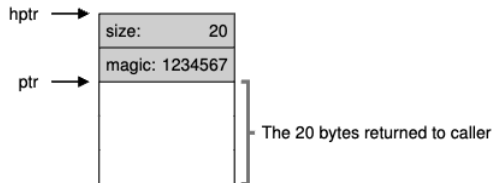
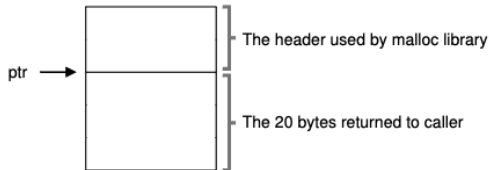
- Ukuran request tidak tetap (kecil, sedang, besar).
- Blok kosong bisa terpecah-pecah.
- Harus cepat, tapi juga hemat ruang.

Trade-off Inti

Waktu alokasi cepat sering berlawanan dengan kualitas tata ruang jangka panjang.

Header Blok pada Heap

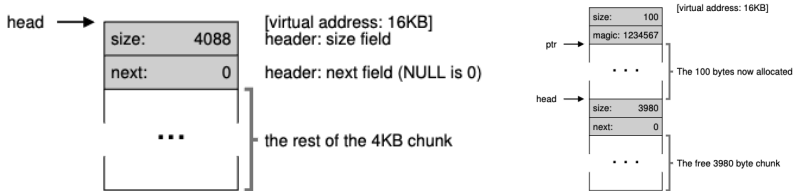
Metadata kecil yang sangat penting



Header menyimpan info seperti ukuran blok dan status (dipakai/kosong), agar `free()` dapat bekerja benar.

Perubahan Heap Saat Alokasi Awal

Visual step-by-step sebelum dan sesudah alokasi



1. Allocator menemukan blok kosong.
2. Jika terlalu besar, blok di-*split*.
3. Bagian yang dipakai diberikan ke user.
4. Sisanya tetap masuk free list.

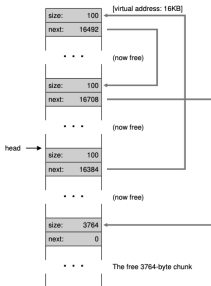
Strategi Fit: First, Best, Worst

Kebijakan memilih blok kosong

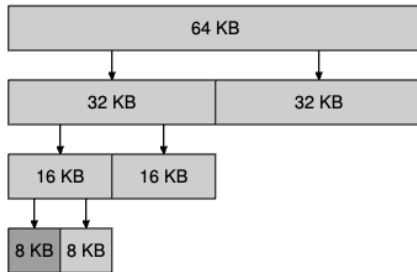
Strategi	Kelebihan	Kekurangan
First Fit	Cepat, sederhana	Cenderung cepat membuat pecahan blok
Best Fit	Sisa blok lokal kecil	Proses pencarian lebih lama
Worst Fit	Menyisakan blok besar	Tidak selalu stabil di praktik

Coalescing dan Buddy System

Dua cara menjaga kualitas free-space



Coalescing: gabung blok kosong bertetangga agar request besar tetap mungkin.



Buddy: pembagian terstruktur (umumnya pangkat dua), merge lebih cepat tapi bisa internal fragmentation.

Checkpoint 5

Penutup bagian allocator

1. Kenapa free list tanpa coalescing bisa jadi buruk?
2. Kapan best fit terlihat menarik? Kapan justru merugikan?
3. Apa trade-off utama buddy system?

Video Pendukung (Free-Space Management)

Cocok ditonton di rumah

1. Memory Allocation and Free Space Management Algorithms

https://youtu.be/bq3_uT5Z424

2. Playlist OS (cadangan belajar mandiri)

<https://www.youtube.com/playlist?list=PLDW872573QAb4bj0URobvQTD41IV6gRkx>

Ringkasan Super Singkat (6 Kalimat Kunci)

Jika lupa materi, ingat 6 poin ini

1. VM memberi ilusi memori privat per proses.
2. Translasi alamat dilakukan hardware sangat sering.
3. Base-and-bounds memberi proteksi cepat dan sederhana.
4. Segmentasi menambah fleksibilitas tata letak memori.
5. Free-space management menjaga alokasi ukuran variabel tetap sehat.
6. Semua desain memori adalah trade-off performa vs efisiensi ruang.

Latihan Mandiri Step-by-Step

Kerjakan pelan, diskusikan berpasangan

1. Diketahui `base=5000`, `bounds=700`. Apakah `v=650` valid? Jika valid, berapa alamat fisiknya?
2. Sebutkan 2 contoh bug heap dan cara pencegahannya.
3. Jelaskan kenapa segmentasi dapat mengurangi pemborosan, tetapi tetap berpotensi fragmentasi eksternal.
4. Jika blok kosong: 12KB, 7KB, 4KB dan request 3KB, strategi fit mana yang Anda pilih? Jelaskan alasan.

Terima Kasih

Ada Pertanyaan?

Ulangi kembali 5 checkpoint tadi. Jika 1 checkpoint masih belum paham, mulai dari bagian itu.